

UNIVERSIDADE SÃO JUDAS TADEU

CURSO DE ENGENHARIA DE SOFTWARE

DISCIPLINA: SISTEMAS AUTOMATIZADOS

KAUÊ MELO DINIZ

RA: 823130975

SISTEMA AUTOMATIZADO DE PORTÃO DE GARAGEM:

Desenvolvimento e Simulação de Lógica Ladder em CLP

Trabalho acadêmico (A3) apresentado à disciplina de Sistemas Automatizados do curso de Engenharia de Software da Universidade São Judas Tadeu, como requisito parcial para avaliação.

Orientador: Prof. Robson Calvetti.

Resumo

Este trabalho apresenta o desenvolvimento e a simulação de um sistema automatizado de portão de garagem controlado por Controlador Lógico Programável (CLP), utilizando a linguagem de programação Ladder (LD). O sistema foi projetado para operar com um único botão de comando, à semelhança de um controle remoto, e contempla duas camadas de segurança — fotocélula de detecção de obstáculos e botão de parada de emergência — além de fechamento automático temporizado. A lógica foi construída e simulada na plataforma PLC Fiddle e organizada em dezoito linhas (rungs), empregando recursos como detecção de borda, memórias do tipo Set/Reset, intertravamento de segurança, temporizador on-delay e instantâneos de estado. Foram definidos oito cenários de teste que cobrem todo o comportamento esperado do portão, e todos foram validados com sucesso, tanto em um simulador independente da lógica quanto no próprio simulador da plataforma. Durante o desenvolvimento foram identificados e corrigidos problemas relacionados à ordem de execução dos rungs e à disputa entre comandos de Set e Reset no mesmo ciclo de varredura, cujas soluções constituem as principais contribuições práticas do trabalho. Conclui-se que a linguagem Ladder, combinada a boas práticas de intertravamento e de tratamento de estados, é adequada e suficiente para implementar de forma segura e confiável a automação proposta.

Palavras-chave: Automação industrial. CLP. Linguagem Ladder. Portão automático. Sistemas de segurança.

Sumário

1 Introdução	5
1.1 Justificativa	5
1.2 Objetivos	6
2 Fundamentação Teórica	7
2.1 Controlador Lógico Programável (CLP)	7
2.2 Linguagem Ladder (LD)	7
2.3 Contatos e bobinas	7
2.4 Temporizadores	8
2.5 Detecção de borda	8
2.6 Segurança em portões automáticos	8
3 Descrição do Sistema Automatizado	9
4 Modelagem do Sistema	10
4.1 Ferramenta de simulação	10
4.2 Arquitetura e metodologia	10
4.3 Mapeamento de entradas e saídas	10
5 Descrição da Lógica de Controle	11
5.1 Estrutura: os dezoito rungs	11
5.2 Conceitos especiais empregados na lógica	14
6 Resultados da Simulação	15
7 Discussão dos Resultados	16
8 Conclusão	17
9 Referências	18
10 Acessos à Simulação	19

1. Introdução

A automação de processos é um dos pilares da engenharia moderna, presente tanto em ambientes industriais de grande porte quanto em aplicações residenciais do cotidiano. Entre os dispositivos mais difundidos para essa finalidade está o Controlador Lógico Programável (CLP), equipamento robusto e flexível capaz de executar rotinas de controle em tempo real a partir da leitura de sensores e do acionamento de atuadores.

Um exemplo bastante familiar de automação residencial é o portão de garagem automático. Embora pareça simples ao usuário — que normalmente interage com ele por meio de um único botão de controle remoto —, o comportamento esperado do portão exige uma lógica de controle relativamente rica: é preciso decidir, a cada acionamento, se o portão deve abrir, fechar, parar ou retomar o movimento, além de tratar situações de segurança como a presença de obstáculos e a necessidade de parada de emergência.

Este trabalho aborda justamente esse problema: como projetar, em linguagem Ladder, a lógica de controle de um portão de garagem que se comporte de forma segura e previsível diante de todas essas situações, utilizando um único botão de comando. O desenvolvimento e a validação foram realizados na plataforma de simulação PLC Fiddle, que reproduz o ciclo de varredura de um CLP e permite testar a lógica sem a necessidade de hardware físico.

A escolha de um sistema acionado por botão único, em vez de dois botões separados para abrir e fechar, foi proposital: ela aproxima o projeto do funcionamento real de um controle remoto e demanda o uso de técnicas mais avançadas de programação Ladder — como detecção de borda, memórias de estado e intertravamentos —, evidenciando de forma mais clara o domínio do conteúdo da disciplina.

1.1 Justificativa

O tema é relevante porque reúne, em um único projeto de porte adequado a um trabalho acadêmico, diversos conceitos fundamentais da automação: leitura de sensores digitais, acionamento de cargas, temporização, memórias retentivas, intertravamento de segurança e tratamento de estados. Além disso, trata-se de uma aplicação de fácil compreensão e de grande apelo prático, o que facilita a verificação do comportamento esperado e a demonstração dos resultados.

1.2 Objetivos

O objetivo geral deste trabalho é projetar, implementar e validar, em linguagem Ladder e em ambiente de simulação, a lógica de controle de um portão de garagem automático acionado por botão único, dotado de recursos de segurança e de fechamento automático.

Como objetivos específicos, destacam-se:

- Levantar as entradas e saídas necessárias ao sistema e definir as memórias internas e o temporizador empregados;
- Especificar o comportamento esperado do portão em todas as situações de operação e de segurança;
- Implementar a lógica em linguagem Ladder na plataforma PLC Fiddle;
- Definir cenários de teste que cubram o comportamento especificado e validá-los por simulação;
- Documentar os problemas encontrados durante o desenvolvimento e as soluções adotadas.

2. Fundamentação Teórica

2.1 Controlador Lógico Programável (CLP)

O Controlador Lógico Programável é um equipamento eletrônico digital de uso industrial que executa, de forma cíclica e contínua, um programa de controle armazenado em sua memória. Seu funcionamento baseia-se no chamado ciclo de varredura (scan), que consiste, de modo simplificado, em três etapas repetidas indefinidamente: leitura das entradas, execução da lógica do programa e atualização das saídas. A cada ciclo, o CLP lê o estado dos sensores conectados às suas entradas, processa a lógica programada e energiza ou desenergiza os atuadores ligados às suas saídas.

Uma característica importante do ciclo de varredura, e que tem impacto direto neste projeto, é a ordem de execução das instruções: as linhas do programa são processadas sequencialmente, de cima para baixo. Para as memórias internas, o valor alterado por uma linha já é enxergado pelas linhas seguintes dentro do mesmo ciclo. Esse comportamento exige atenção quando uma mesma condição comanda o ligar e o desligar de uma mesma memória, como será discutido na Seção 5.

2.2 Linguagem Ladder (LD)

A linguagem Ladder, ou diagrama de contatos, é uma das linguagens de programação de CLPs padronizadas pela norma IEC 61131-3. Sua representação gráfica é inspirada nos antigos diagramas elétricos a relés: a lógica é desenhada entre duas barras verticais de alimentação e organizada em linhas horizontais, chamadas de degraus ou rungs. A energia, de forma simbólica, flui da barra esquerda para a direita, atravessando contatos até chegar às bobinas de saída.

Sua ampla adoção deve-se à facilidade de leitura por profissionais de manutenção elétrica e à correspondência direta com a lógica de relés, o que torna a linguagem intuitiva para representar intertravamentos e sequências de comando.

2.3 Contatos e bobinas

Os contatos representam as condições lógicas (entradas) de cada linha. O contato normalmente aberto (NA) conduz quando a variável associada é verdadeira; o contato normalmente fechado (NF) conduz quando a variável é falsa, ou seja, opera de forma

invertida. Contatos em série representam a operação lógica E (AND), enquanto contatos em caminhos paralelos representam a operação OU (OR).

As bobinas representam as saídas ou ações da linha. A bobina simples (Output) assume, a cada ciclo, o valor da continuidade lógica que chega até ela. Já as bobinas retentivas Set (ou Latch) e Reset (ou Unlatch) funcionam aos pares: a bobina Set liga a memória e a mantém ligada mesmo que a condição deixe de ser verdadeira, ao passo que a bobina Reset a desliga. Esse par é fundamental para construir lógicas de estado, como o registro de que um comando está ativo.

2.4 Temporizadores

O temporizador on-delay (TON) é uma instrução que introduz um atraso controlado. Enquanto sua entrada de habilitação permanece verdadeira, o temporizador acumula tempo; quando o tempo acumulado (ACC) atinge o valor de preset (PRE), o bit de saída (Q, ou “timer done”) torna-se verdadeiro. Se a habilitação for interrompida antes de o tempo se esgotar, o acumulador é zerado. Neste projeto, o temporizador é utilizado para implementar o fechamento automático do portão após um intervalo parado aberto.

2.5 Detecção de borda

Botões de comando momentâneos permanecem acionados por vários ciclos de varredura enquanto o usuário mantém o dedo pressionado. Para que cada acionamento produza uma única ação — e não uma sequência de ações a cada ciclo —, emprega-se a técnica de detecção de borda de subida, que gera um pulso de duração de apenas um ciclo no instante em que a entrada passa de falsa para verdadeira. Isso é obtido comparando-se o valor atual da entrada com o valor que ela tinha no ciclo anterior, armazenado em uma memória auxiliar.

2.6 Segurança em portões automáticos

Portões automáticos são equipamentos que oferecem risco de esmagamento e, por isso, devem incorporar dispositivos de segurança. Os mais comuns são a fotocélula — uma barreira óptica que detecta a presença de pessoas, veículos ou objetos no vão do portão — e a parada de emergência. A boa prática determina que, ao detectar um obstáculo durante o fechamento, o portão interrompa imediatamente o movimento e

reabra, dando prioridade à abertura até completar o curso. Este projeto adota essa filosofia, tratando a detecção de obstáculo como uma trava de segurança que só é liberada quando o portão volta a atingir a posição totalmente aberta.

3 Descrição do Sistema Automatizado

O sistema controla um portão de garagem acionado por um único botão, à semelhança de um controle remoto. O comportamento especificado é o seguinte: com o portão fechado, um acionamento do botão inicia a abertura; um novo acionamento durante o movimento faz o portão parar na posição em que estiver; um acionamento adicional, com o portão parado no meio do curso, retoma o mesmo movimento que vinha sendo executado. Com o portão totalmente aberto, um acionamento inicia o fechamento. Caso permaneça aberto e sem comando por dez segundos, o portão fecha automaticamente. Se a fotocélula detectar um obstáculo durante o fechamento, o motor de fechamento é interrompido de imediato e o portão reabre por segurança. Por fim, enquanto o botão de emergência estiver acionado, nenhum motor é energizado; ao liberar a emergência, o portão permanece parado, voltando a operar apenas com um novo acionamento do botão.

O sistema é composto, do ponto de vista físico, por um conjunto de sensores de entrada (o botão de comando, dois fins de curso que indicam as posições totalmente aberta e totalmente fechada, a fotocélula de segurança e o botão de emergência) e por um conjunto de atuadores de saída (o motor, comandado em dois sentidos por contadores independentes, e um sinaleiro de advertência). Toda a inteligência do processo reside na lógica programada no CLP, que interpreta os sensores e decide o acionamento dos atuadores conforme o estado atual do portão.

4 Modelagem do Sistema

4.1 Ferramenta de simulação

A modelagem e os testes foram realizados na plataforma PLC Fiddle (www.plcfiddle.com), simulador on-line de lógica Ladder recomendado na especificação do projeto e voltado a aprendizado, treinamento e compartilhamento de código. A ferramenta permite criar variáveis (booleanas, numéricas, temporizadores e contadores), montar os rungs por meio do arrastar de contatos e bobinas, e executar a simulação em tempo real, alterando manualmente o estado das entradas e observando a resposta das saídas. Por dispensar hardware físico, mostrou-se adequada ao escopo do trabalho.

4.2 Arquitetura e metodologia

A modelagem seguiu uma sequência de etapas bem definida. Primeiramente, especificou-se o comportamento esperado do portão em todas as situações de uso e de segurança (Seção 3). Em seguida, levantou-se a lista de entradas, saídas, memórias internas e o temporizador necessários, apresentada na Seção 4.3. A partir dessa modelagem, a lógica foi implementada rung a rung na plataforma e, por fim, validada por meio de oito cenários de teste derivados diretamente do comportamento especificado. Para aumentar a confiabilidade da validação, além dos testes manuais no próprio PLC Fiddle, a mesma lógica foi verificada de forma independente por um simulador de varredura desenvolvido à parte, e os resultados das duas abordagens foram comparados.

4.3 Mapeamento de entradas e saídas

Quadro 1 — Entradas físicas

Tag	Tipo	Descrição
I_Botao	NA	Botão único de controle (momentâneo)
I_FimAberto	NA	Fim de curso — portão 100% aberto
I_FimFechado	NA	Fim de curso — portão 100% fechado
I_Fotocelula	NA	Barreira de segurança (1 = obstáculo)
I_Emergencia	NA	Botão de parada de emergência (1 = acionado)

Quadro 2 — Saídas físicas

Tag	Descrição
Q_MotorAbrir	Contator do motor no sentido de abertura
Q_MotorFechar	Contator do motor no sentido de fechamento
Q_Sinaleiro	Luz de alerta, acesa durante o movimento

Quadro 3 — Memórias internas e temporizador

Tag	Função
M_Habilitado	Libera os motores (verdadeiro = sem emergência)
M_CmdAbrir	Comando interno de abertura ativo
M_CmdFechar	Comando interno de fechamento ativo
M_Obstaculo	Trava de segurança: obstáculo detectado ao fechar
M_UltimoAbrir	Memória da última direção comandada
M_BotaoAnterior	Valor de I_Botao no ciclo anterior (borda)
M_PulsoBotao	Pulso de 1 ciclo gerado ao apertar o botão
M_EstavaAbrindo	Instantâneo: estava abrindo no início do ciclo
M_EstavaFechando	Instantâneo: estava fechando no início do ciclo
T_FechAuto	Temporizador TON, preset de 10 s (fechamento automático)

5 Descrição da Lógica de Controle

5.1 Estrutura: os dezoito rungs

A lógica completa foi organizada em dezoito rungs, apresentados no Quadro 4 na mesma ordem em que são executados pelo CLP. A notação adotada é: NA para contato normalmente aberto, NF para normalmente fechado, () para bobina simples, (L) para Set/Latch e (U) para Reset/Unlatch; caminhos paralelos (operação OU) são indicados por alíneas a, b e c.

Diagrama Ladder da lógica de controle

Quadro 4 — Linhas (rungs) da lógica em ordem de execução

#	Condição	Bobina
1	NF I_Emergencia	(M_Habilitado)
2	NA I_Emergencia	(U M_CmdAbrir) e (U M_CmdFechar)
3	NA I_Botao + NF M_BotaoAnterior	(M_PulsoBotao)
4	NA I_Botao	(M_BotaoAnterior)
5	NA M_CmdFechar + NA I_Fotocelula	(L M_Obstaculo)
6	NA I_FimAberto	(U M_Obstaculo)
7	NA M_CmdAbrir	(L M_UltimoAbrir)
8	NA M_CmdFechar	(U M_UltimoAbrir)
9	NA M_CmdAbrir	(M_EstavaAbrindo)
10	NA M_CmdFechar	(M_EstavaFechando)
11	a) NA M_PulsoBotao + NA M_CmdAbrir b) NA I_FimAberto	(U M_CmdAbrir)
12	a) NA M_PulsoBotao + NA M_CmdFechar b) NA I_FimFechado c) NA I_Fotocelula + NA M_CmdFechar	(U M_CmdFechar)
13	a) NA M_PulsoBotao + NA M_Habilitado + NA I_FimFechado b) NA M_Obstaculo c) NA M_PulsoBotao + NA M_Habilitado + NF I_FimAberto + NF I_FimFechado + NF M_CmdAbrir + NF M_CmdFechar + NA M_UltimoAbrir + NF M_EstavaAbrindo	(L M_CmdAbrir)
14	a) NA M_PulsoBotao + NA M_Habilitado + NA I_FimAberto + NF I_Fotocelula b) NA T_FechAuto.Q + NA I_FimAberto + NF I_Fotocelula + NF M_Obstaculo c) NA M_PulsoBotao + NA M_Habilitado + NF I_FimAberto + NF I_FimFechado + NF M_CmdAbrir + NF M_CmdFechar + NF M_UltimoAbrir + NF I_Fotocelula + NF M_EstavaFechando	(L M_CmdFechar)
15	NA I_FimAberto + NF I_Fotocelula	[TON T_FechAuto, 10 s]
16	NA M_CmdAbrir + NA M_Habilitado	(Q_MotorAbrir)
17	NA M_CmdFechar + NA M_Habilitado	(Q_MotorFechar)
18	NA M_CmdAbrir OU NA M_CmdFechar	(Q_Sinaleiro)

A título de explicação, os rungs podem ser agrupados por função. O rung 1 estabelece a habilitação geral a partir do botão de emergência. O rung 2 zera os comandos internos enquanto a emergência estiver acionada, garantindo que o portão não volte a se mover sozinho ao liberá-la. Os rungs 3 e 4 implementam a detecção de borda do botão. Os rungs 5 e 6 controlam a trava de obstáculo, e os rungs 7 e 8 registram a última direção de movimento. Os rungs 9 e 10 capturam, no início de cada ciclo, se o

portão estava em movimento. Os rungs 11 e 12 desligam (Reset) os comandos de abrir e fechar; os rungs 13 e 14, logo em seguida, ligam (Set) esses comandos. Por fim, o rung 15 controla o temporizador de fechamento automático e os rungs 16 a 18 acionam, respectivamente, o motor de abertura, o motor de fechamento e o sinaleiro.

5.2 Conceitos especiais empregados na lógica

Três decisões de projeto merecem destaque por serem responsáveis pelo funcionamento correto do sistema.

A primeira é o pulso do botão. Como o botão é momentâneo, sua leitura direta em nível faria com que cada acionamento provocasse múltiplas ações enquanto o usuário mantivesse o dedo pressionado. A detecção de borda (rungs 3 e 4) resolve isso gerando um pulso de apenas um ciclo a cada acionamento.

A segunda é o instantâneo de movimento, representado pelas memórias M_EstavaAbrindo e M_EstavaFechando (rungs 9 e 10). Como o mesmo pulso do botão comanda tanto o parar quanto o retomar de um movimento, e como o CLP atualiza as memórias dentro do mesmo ciclo, surge uma disputa: no ciclo em que o comando é desligado pelo Reset, o rung de Set logo abaixo enxergaria esse comando já desligado e o religaria, de modo que o portão nunca pararia. Para resolver, capturam-se no início do ciclo, antes dos rungs de Reset e Set, os estados dos comandos. Assim, o parar passa a depender de “estava se movendo” e o retomar de “estava parado”, condições mutuamente exclusivas que eliminam a disputa

A terceira é a ordem de execução dos rungs. Para que a lógica funcione, os Resets dos comandos (rungs 11 e 12) precisam ser executados antes dos respectivos Sets (rungs 13 e 14), e os instantâneos (rungs 9 e 10) precisam vir antes de ambos. Essa ordenação é o que permite que o instantâneo guarde o valor verdadeiro do início do ciclo.

6 Resultados da Simulação

Para validar a lógica, foram definidos oito cenários de teste derivados diretamente do comportamento especificado na Seção 3. O Quadro 5 apresenta cada cenário, o resultado esperado e o resultado obtido.

Quadro 5 — Cenários de teste e resultados

#	Cenário	Resultado esperado	Resultado
1	Fechado, aperta o botão	Q_MotorAbrir liga até o fim aberto	Aprovado
2	Abrindo, aperta o botão	Q_MotorAbrir desliga (para no meio)	Aprovado
3	Parado no meio, aperta o botão	Q_MotorAbrir religa (retoma), estável	Aprovado
4	Aberto, aperta o botão	Q_MotorFechar liga até o fim fechado	Aprovado
5	Fechando, fotocélula detecta	Para de fechar e reabre por segurança	Aprovado
6	Aberto e parado por 10 s	Q_MotorFechar liga sozinho	Aprovado
7	Emergência acionada	Ambos os motores desligam	Aprovado
8	Emergência liberada	Permanece parado até novo acionamento	Aprovado

Os oito cenários foram aprovados. A validação foi conduzida de duas formas complementares: a primeira, manualmente, no simulador do PLC Fiddle, alterando o estado das entradas e observando as saídas; a segunda, de forma automatizada, por meio de um simulador de ciclo de varredura desenvolvido à parte, que executa a mesma lógica e verifica os resultados de cada cenário. As duas abordagens apresentaram resultados idênticos, o que reforça a confiabilidade da validação.

Um aspecto prático observado durante os testes diz respeito ao acionamento do botão no simulador: como a ação é disparada pela borda de subida da entrada, o efeito ocorre no instante em que o botão passa de desligado para ligado. Acionamentos sucessivos e rápidos alternam, portanto, entre parar e retomar o movimento, comportamento coerente com o de um controle remoto real.

7 Discussão dos Resultados

A aprovação dos oito cenários confirma que a lógica atende integralmente ao comportamento especificado. Mais do que o resultado final, o desenvolvimento iterativo revelou situações que não eram evidentes na especificação inicial e cuja análise constitui parte importante do aprendizado obtido com o trabalho. Esta seção discute os principais problemas encontrados e as soluções adotadas.

O primeiro problema foi a disputa entre Set e Reset compartilhando o mesmo pulso do botão. Inicialmente, o portão não abria ao primeiro acionamento, pois o rung de Reset, posicionado depois do de Set, enxergava o comando recém-ligado e o desligava no mesmo ciclo. A correção consistiu em executar o Reset antes do Set.

O segundo problema surgiu como efeito da própria correção anterior: com a nova ordem, o portão deixava de parar ao se apertar o botão durante o movimento, pois o rung de Set religava o comando que o Reset acabara de desligar. A solução definitiva foi a introdução dos instantâneos M_EstavaAbrindo e M_EstavaFechando, descritos na Seção 5.2, que tornam o parar e o retomar mutuamente exclusivos independentemente da ordem dos rungs.

O terceiro problema referia-se ao caminho do fechamento automático, cuja bobina, por engano, acionava diretamente a saída do motor de fechamento em vez do comando interno correspondente, contornando a habilitação de segurança e deixando a saída travada. A correção redirecionou a bobina para o comando interno M_CmdFechar.

O quarto ponto foi o ajuste do preset do temporizador, definido em dez segundos, observando-se que, na plataforma utilizada, o preset é expresso em segundos. O quinto e último foi a inclusão do rung que zera os comandos durante a emergência, sem o qual o portão retomava o movimento sozinho ao se liberar a parada de emergência. Todos os problemas foram corrigidos e reverificados, levando à aprovação dos oito cenários.

8 Conclusão

Este trabalho desenvolveu e validou, em linguagem Ladder e em ambiente de simulação, a lógica de controle de um portão de garagem automático acionado por botão único, atendendo a todos os objetivos propostos. A solução final, organizada em dezoito rungs, implementa de forma segura o conjunto completo de comportamentos especificados, incluindo abertura, fechamento, parada e retomada de movimento, reabertura por detecção de obstáculo, fechamento automático temporizado e parada de emergência.

Os oito cenários de teste definidos foram integralmente aprovados, validados de duas formas independentes que apresentaram resultados coincidentes. Mais do que o resultado final, o processo de desenvolvimento evidenciou conceitos essenciais da programação de CLPs, em especial a importância da ordem de execução dos rungs e o tratamento cuidadoso de memórias de estado quando uma mesma condição comanda ações opostas.

Conclui-se que a linguagem Ladder, aliada a boas práticas de detecção de borda, intertravamento e tratamento de estados, é plenamente adequada para implementar de modo confiável e seguro a automação proposta. Como trabalhos futuros, sugere-se a implementação física em um CLP real, a inclusão de sinalização sonora e a adição de um sensor de corrente no motor como camada adicional de detecção de obstáculos.

9 Referências

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

FRANCHI, Claiton Moro; CAMARGO, Valter Luís Arlindo de. Controladores lógicos programáveis: sistemas discretos. 2. ed. São Paulo: Érica, 2009.

INTERNATIONAL ELECTROTECHNICAL COMMISSION. IEC 61131-3: programmable controllers: part 3: programming languages. Genebra: IEC, 2013.

PETRUZELLA, Frank D. Controladores lógicos programáveis. 4. ed. Porto Alegre: AMGH, 2014.

PLC FIDDLE. The online ladder logic simulator for testing, training, and code sharing. Disponível em: <https://www.plcfiddle.com>. Acesso em: 23 jun. 2026.

PRUDENTE, Francesco. Automação industrial: PLC: teoria e aplicações: curso básico. 2. ed. Rio de Janeiro: LTC, 2011.

10 Acesso à Simulação

A lógica Ladder completa, montada e validada na plataforma PLC Fiddle, pode ser acessada e simulada publicamente no seguinte endereço:

<https://www.plcfiddle.com/fiddles/4c5b9150-3a33-47d7-8011-9ba8750eeeb1>

Para reproduzir os testes, basta abrir o endereço, garantir que o portão esteja na condição inicial fechada (I_FimFechado ligado) e acionar as entradas conforme os cenários do Quadro 5, observando o comportamento das saídas Q_MotorAbrir, Q_MotorFechar e Q_Sinaleiro.